

Types de base et opérateurs associés

- **int** : nombres entiers de taille *non limitée*

+	addition	
-	soustraction	
*	multiplication	
/	division <i>décimale</i>	ex : 13/5 vaut 2.6.
**	exponentiation	ex : 3**4 vaut 81.
//	quotient dans la division euclidienne	ex : 13//5 vaut 2.
%	reste dans la division euclidienne	ex : 13%5 vaut 3. Tests de divisibilité.

- **float** : nombres en virgule flottante de taille limitée
S'écrivent toujours avec le séparateur décimal ., mêmes opérateurs que sur les entiers (à l'exception de // et %).

- **bool** : deux valeurs possibles **True** et **False**

not	négation (unaire)	inverse la valeur de l'argument.
or	ou (binaire)	vaut True si au moins un des arguments vaut True .
and	et (binaire)	vaut True si les deux arguments valent True .

- **str** : chaînes de caractères

+	concaténation	ex : "Hello" + "World" vaut "HelloWorld"
*	répétition	ex : "Euh" * 4 vaut "EuhEuhEuhEuh"
len	longueur	ex : len("Python") vaut 6
[]	i-ème caractère	numérotation depuis 0, ex : "Hello"[1] vaut "e"

Des conversions sont possibles entre ces divers types, par exemple, `int("34")` transforme la chaîne de caractères "34" en l'entier 34, `float(34)` transforme l'entier 34 en flottant 34.0.

Comparaison

>	strictement supérieur	
<	strictement inférieur	
>=	supérieur ou égal	les symboles sont dans l'ordre de leur lecture.
<=	inférieur ou égal	
==	égal	⚠ à ne pas confondre avec = utilisé pour l'affectation des variables.
!=	différent	

Instructions conditionnelles

- Pour exécuter des <instructions> si une condition est vérifiée :

Syntaxe :

```
1 if <condition>:
2     <instructions>
```

Exemple :

```
1 if a != 0:
2     b = 1 / a
```

- Pour exécuter <instructions1> si une condition est vérifiée et <instructions2> sinon :

Syntaxe :

```
1 if <condition>:
2     <instructions1>
3 else:
4     <instructions2>
```

Exemple :

```
1 if x < y:
2     minimum = x
3 else:
4     minimum = y
```

- Pour tester plusieurs conditions exclusives, on enchaîne avec la clause **elif** (*else if*) :

Syntaxe :

```
1 if <cond1>:
2     <inst1>
3 elif <cond2>:
4     <inst2>
5 else:
6     <inst3>
```

Exemple :

```
1 if note >= 16:
2     mention = "TB"
3 elif note >= 14:
4     mention = "B"
5 else:
6     mention = "AB ou passable"
```

Boucles while

Les boucles **while** répètent un bloc d'instructions tant qu'une condition est vraie.

Syntaxe :

```
1 while <condition>:
2     <instructions>
```

Exemple :

```
1 rep = ""
2 while rep != 'O' and rep != 'N':
3     rep = input("O/N ?")
```

Boucles for avec range

- L'instruction **range** génère des entiers et peut prendre un, deux ou trois arguments :

range(n)	génère les n entiers de l'intervalle $\llbracket 0; n - 1 \rrbracket$
range(m,n)	génère les entiers de l'intervalle $\llbracket m; n - 1 \rrbracket$
range(m,n,s)	génère les entiers de l'intervalle $\llbracket m; n - 1 \rrbracket \cap \{m + ks, k \in \mathbb{N}\}$

⚠ Dans les trois cas, la valeur **n** n'est pas prise.

- L'instruction **range** peut s'utiliser conjointement avec une boucle **for** pour créer une variable qui prendra les valeurs générées par le **range**, le bloc d'<instructions> qui suit est alors exécuté pour chaque valeur de la variable.

Syntaxe :

```
1 for <variable> in range(...):
2     <instructions1>
```

Exemple :

```
1 for i in range(1, 9):
2     print(i) # va afficher 1, 2, ... 8
```

Une boucle **for** permet donc en particulier de *répéter* un nombre donné de fois des instructions.

Parcours d'une liste ou d'une chaîne

Par élément (à privilégier si l'indice n'est pas utile) :

```
1 for x in ma_liste:
2     print(x) # x prend la valeur de chaque
```

Par indice (nécessaire pour modifier la liste ou accéder aux voisins) :

```
1 for i in range(len(ma_liste)):
2     print(ma_liste[i]) # accès par indice
```

Fonctions

- Les fonctions sont des blocs d'instructions réutilisables (chaque appel à la fonction exécute son bloc d'instructions), leur définition commence par le mot-clé **def** puis on écrit le nom de la fonction puis la liste de ses arguments entre parenthèses (séparés par des virgules).
- Une fonction peut prendre zéro, un ou plusieurs arguments.
- Une fonction *peut* renvoyer un résultat à l'aide d'une instruction **return**.
- Exemple : une fonction à 3 arguments et qui renvoie une valeur

```
1 def discriminant(a, b, c):  
2     return b**2 - 4*a*c
```

- Exemple : une fonction à un argument et qui ne renvoie rien (elle produit un affichage)

```
1 def triangle(n):  
2     for i in range(1, n):  
3         print("*" * i)
```

⚠ Ne pas confondre **print** et **return**. La fonction **discriminant** ci-dessus *renvoie* un résultat, donc on pourrait écrire `d = discriminant(1, -11, 30)` afin de récupérer dans `d` la valeur calculée. La fonction **triangle** ne renvoie rien, elle produit un affichage, il serait donc illogique d'écrire `t = triangle(4)`. Par contre `triangle(4)` fera appel à cette fonction et affichera un triangle de 3 lignes d'étoiles.

Spécification et assertions

L'instruction **assert** <condition> valide une précondition. Si la condition est fausse, le programme s'arrête immédiatement en levant une exception **AssertionError** :

```
1 def quotient(a, b):  
2     assert b != 0 # Précondition : b doit être non nul  
3     return a // b
```

Tranches

❶ Accès à un caractère par son indice

La notation `[i]` déjà rencontrée sur les chaînes de caractères permet d'accéder au *i*-ème caractère d'une chaîne où les caractères sont numérotés à *partir de 0*. Par exemple, si `exemple = "Un petit exemple"` :

U	n		p	e	t	i	t		e	x	e	m	p	l	e
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

alors : `exemple[0]` est 'U', `exemple[1]` est 'n', ...

❗ On remarquera que l'indice du dernier élément est *la longueur de la chaîne moins 1*. La longueur s'obtenant avec `len`, ici on a par exemple `exemple[len(exemple)-1]` qui vaut 'e'.

❷ Tranches

On peut aussi prendre une tranche en précisant dans les `[]` le début de la tranche (inclus) et sa fin (exclue) séparés par le caractère `:`; si le début ou la fin sont absents alors ils correspondent respectivement au premier et au dernier indice. Par exemple :

- `exemple[3:8]` est "petit"
- `exemple[:2]` est "Un" (le début étant absent, on commence au premier caractère)
- `exemple[13:]` est "ple" (la fin étant absente, on termine au dernier caractère)

❸ Pas de progression

Une tranche peut prendre un troisième paramètre qui indique alors un *pas de progression*, par exemple si ce pas vaut 2, on ne prend qu'un caractère sur 2. D'autre part si le pas est négatif alors on progresse de la fin de la chaîne vers le début. Par exemple :

- `exemple[4:10:2]` est "ei "
- `exemple[15:8:-1]` est "elpmexe"
- `exemple[::-1]` est "elpmexe titep nU" le pas étant négatif on progresse de la fin (absente donc dernier caractère) jusqu'au début (absent donc premier caractère).

Tuples

- ❶ Un tuple est une suite de valeurs repérées par leur indice (à la façon des caractères d'une chaîne). Un tuple se note entre `()` et les valeurs sont séparées par des virgules. Par exemple `date = (2, "décembre", 1815)` est un tuple constitué de trois valeurs.
- ❷ On retrouve pour les tuples, la fonction `len`, l'accès au *i*-ème élément avec `[i]` et les tranches déjà vues sur les chaînes de caractères.
- ❸ Les valeurs d'un tuple ne sont *pas modifiables* (comme les caractères d'une chaîne), une tentative en ce sens produit un `TypeError`.
- ❹ Un tuple peut être dépaqueté afin d'affecter chacune de ses valeurs à une variable. Par exemple `jour, mois, annee = date`.

Importation de fonctions

En Python, on peut importer des fonctions se trouvant dans d'autres modules, deux syntaxes sont possibles :

- `from <module> import <fonction>`, cela rend directement utilisable `<fonction>` dans la suite du programme. Par exemple la fonction racine carrée s'appelle `sqrt` et doit être importée depuis le module `math` avec `from math import sqrt` pour être utilisable.
- `import <module>`, dans ce cas, toutes les fonctions du module sont utilisables mais on doit préfixer leur nom par celui du module. Par exemple après un `import math` pour utiliser la fonction racine carrée, on doit écrire `math.sqrt`.

Listes

- ❶ Une liste est une suite de valeurs repérées par leur indice. Une liste se note entre [et] et les valeurs sont séparées par des virgules. Par exemple `premiers = [2, 3, 5, 7, 11, 13, 15]` est une liste (qui contient une erreur en dernière position). La liste vide est `[]`.
- ❷ On retrouve pour les listes, la fonction `len`, l'accès au *i*-ème élément avec `[i]` et les tranches déjà vues sur les chaînes de caractères et les tuples.
- ❸ Les valeurs d'une liste, à la différence de celles d'un tuple, sont *modifiables*, on peut donc écrire `premiers[6] = 17` afin que la liste ci-dessus devienne `premiers = [2, 3, 5, 7, 11, 13, 17]`.
- ❹ On peut ajouter un élément à une liste avec `append`, la syntaxe est `<liste>.append(<element>)`. Par exemple, après exécution de `premiers.append(19)` la liste ci-dessus devient `premiers = [2, 3, 5, 7, 11, 13, 17, 19]`.
- ❺ On peut retirer le dernier élément d'une liste avec `pop`, la syntaxe est `<liste>.pop()`. L'élément retiré est renvoyé par cette instruction et peut être récupéré, ainsi `n = premiers.pop()` aura deux effets : supprimer 19 de la liste `premiers` et affecter cette valeur à `n`.
- ❻ Création de listes :
 - en donnant explicitement ses éléments (comme la liste `premiers` ci-dessus).
 - par répétition avec `*`, par exemple `[77] * 10` est la liste constituée de 10 fois le nombre 77.
 - par ajouts successifs, on part d'une liste vide et on ajoute (généralement à l'aide d'une boucle `for`) successivement avec `append` les éléments à la liste.
 - par compréhension, à la façon dont on définit parfois les ensembles en mathématiques. Par exemple, `[i for i in range(50) if i % 10 == 7]` est la liste `[7, 17, 27, 37, 47]` (les nombres entre 0 et 49 dont le reste dans la division euclidienne par 10 est 7).

Mutables et non mutables

En Python, on manipule des objets, certains sont *mutables* et d'autres non. Une variable est l'association entre une étiquette (le nom) et l'objet qu'elle référence. Les listes de Python sont mutables au contraire de tous les autres types rencontrés jusqu'ici (`int`, `float`, `bool`, `str`, `tuple`). Lorsqu'on modifie un objet mutable, cela modifie toutes ses *références*.

Cas non mutable :

```
1 n = 42
2 m = n
3 m = m + 1
4 # n vaut toujours 42
5 # m et n référencent deux objets diffé
```

Cas mutable :

```
1 n = [12, 15]
2 m = n
3 m.append(17)
4 # m référence maintenant [12, 15, 17]
5 # m et n référencent le même objet
```

Les fonctions qui modifient un objet mutable le font généralement sans faire apparaître le signe `=`, par exemple `liste.pop()`.

Fichiers textes

- **Lecture** : `f = open("data.txt", "r")`, puis `f.read()` (tout le contenu sous forme de `str`) ou `f.readline()` (une ligne), puis fermeture `f.close()`.
- **Écriture** : `f = open("sortie.txt", "w")`, puis `f.write("texte\n")`, puis fermeture `f.close()`. (Le mode `"a"` permet d'écrire en ajout sans écraser).
- Découpage d'une ligne lue avec séparateur : `mots = ligne.split(";")`.

**Mémo Algorithmique : Terminaison, Correction, Complexité**

- **Terminaison** : Se démontre à l'aide d'un **variant** (quantité entière positive qui décroît strictement à chaque passage dans la boucle non bornée ou appel récursif).
- **Correction** : Se démontre à l'aide d'un **invariant de boucle** (propriété vraie avant la boucle et conservée à chaque itération) ou par récurrence sur les appels récursifs.
- **Complexités usuelles** ($\mathcal{O}$) : Majoration asymptotique du nombre d'opérations dans le pire cas :

$$\mathcal{O}(1) \ll \mathcal{O}(\log n) \ll \mathcal{O}(n) \ll \mathcal{O}(n \log n) \ll \mathcal{O}(n^2) \ll \mathcal{O}(2^n)$$